

Introducción a Programación Funcional con TypeScript

Guía de Estudio — Tema 03

Matías Gel

Ciclo lectivo 2026

Índice de Contenidos

1	Guía de Estudio — Tema 03	2
1.1	Introducción a Programación Funcional con TypeScript	2
1.2	Cómo usar esta guía	2
1.3	Objetivos de Aprendizaje	2
1.4	Conocimientos Previos	4
1.5	Parte I — Contexto Histórico y Origen del Paradigma	4
1.5.1	1.1 El problema que lo originó todo: el Entscheidungsproblem . . .	4
1.5.2	1.2 Línea de tiempo de lenguajes funcionales	5
1.6	Parte II — El Modelo Funcional de Computación	6
1.6.1	2.1 Dos modelos, dos filosofías	6
1.6.2	2.2 Ejemplo concreto: mismo problema, dos paradigmas	6
1.6.3	2.3 β -reducción: el mecanismo del λ -cálculo	7
1.7	Parte III — Los Tres Pilares del Paradigma Funcional	8
1.7.1	3.1 Pilar 1: Funciones Puras	8
1.7.2	3.2 Pilar 2: Inmutabilidad	9
1.7.3	3.3 Pilar 3: Transparencia Referencial	10
1.8	Parte IV — TypeScript en Modo Funcional	10
1.8.1	4.1 map — transformar sin mutar	10
1.8.2	4.2 filter — seleccionar sin mutar	11
1.8.3	4.3 reduce — agregar sin mutar	11
1.8.4	4.4 Composición con pipe	12
1.8.5	4.5 Funciones de Orden Superior	13
1.9	Parte V — Clojure: el Funcional Puro	13
1.9.1	5.1 ¿Qué es Clojure?	13
1.9.2	5.2 Sintaxis básica de Clojure	14
1.9.3	5.3 map, filter, reduce en Clojure	14
1.9.4	5.4 Comparativa TypeScript vs Clojure	15
1.10	Parte VI — Lenguajes Puros vs Multiparadigma	16
1.10.1	6.1 Lenguajes funcionales puros	16
1.10.2	6.2 El funcional en lenguajes multiparadigma	16
1.11	Puntos Clave del Tema	16

1.12 Autoevaluación	17
1.12.1 Nivel 1 — Comprensión conceptual	17
1.12.2 Nivel 2 — Aplicación	18
1.12.3 Nivel 3 — Análisis y argumentación	18
1.12.4 Respuestas de Autoevaluación	19
1.13 Glosario	20
1.14 Referencias Bibliográficas	22

1 Guía de Estudio — Tema 03

1.1 Introducción a Programación Funcional con TypeScript

Materia: Paradigmas y Lenguajes de Programación 2026 — UNTDF / IDEI
 — IF020 **Año:** 4° Licenciatura en Sistemas **Semana:** 2 — Clase 1 **Duración**
estimada de estudio autónomo: 3-4 horas

1.2 Cómo usar esta guía

Esta guía está diseñada para ser **autocontenida**: no es necesario abrir el libro de texto para estudiar el tema. El contenido está extraído directamente de las fuentes de la cátedra (Gabbrielli-Martini 2023, cap. 11; Sebesta 2019, cap. 15) y del apunte UNTDF 2025.

Estructura sugerida de lectura: 1. Leer la sección de contexto (fondo conceptual) 2. Trabajar los ejemplos de código — copiarlos y ejecutarlos 3. Responder las preguntas de autoevaluación sin mirar las respuestas 4. Usar el glosario para consolidar terminología

1.3 Objetivos de Aprendizaje

Al finalizar el estudio de este tema, debés ser capaz de:

#	Nivel Bloom	Objetivo
OA-1	Comprender	Explicar la diferencia fundamental entre computación por transformación de estado vs computación por reescritura de expresiones
OA-2	Comprender	Enunciar los tres pilares del paradigma funcional: funciones puras, inmutabilidad y transparencia referencial
OA-3	Aplicar	Escribir funciones puras en TypeScript usando const, arrow functions y sin efectos colaterales
OA-4	Aplicar	Usar map, filter y reduce en TypeScript como sustitutos funcionales de los loops imperativos
OA-5	Analizar	Justificar por qué cada restricción funcional en TS (no let, no mutación, no loops) mejora predecibilidad del código
OA-6	Analizar	Reconocer en Clojure la expresión pura de los mismos conceptos (listas, map, filter, funciones anónimas con fn)

#	Nivel Bloom	Objetivo
OA-7	Comprender	Trazar el origen del paradigma funcional desde el λ -cálculo de Church y explicar por qué los lenguajes multiparadigma incorporaron características funcionales a partir de mediados de los 2000s

1.4 Conocimientos Previos

Este tema da por descontado que ya manejas:

- Variables, control de flujo y loops en cualquier lenguaje imperativo (Tema 01)
- Concepto de función como unidad de abstracción
- Estructuras de datos básicas: arrays, objetos
- Noción elemental de TypeScript (tipos estáticos, const, arrow functions =>)

Si alguno de estos puntos te genera dudas, repasá el material de Tema 01 antes de continuar.

1.5 Parte I — Contexto Histórico y Origen del Paradigma

1.5.1 1.1 El problema que lo originó todo: el Entscheidungsproblem

En 1928, David Hilbert formuló el **Entscheidungsproblem** (problema de la decisión):

“¿Existe un procedimiento mecánico que, dado cualquier enunciado matemático, determine de forma finita si es verdadero o falso?”

En 1936 llegaron dos respuestas simultáneas e independientes, ambas negativas:

	Alonzo Church (Princeton)	Alan Turing (Cambridge)
Herramienta	λ -cálculo — sistema formal de funciones y sustitución	Máquina de Turing — dispositivo abstracto con estados y cinta
Resultado	No existe tal algoritmo	No existe tal algoritmo (halting problem)
Legado	→ Paradigma funcional	→ Paradigma imperativo / von Neumann

La Tesis Church-Turing establece que ambos modelos son computacionalmente equivalentes: todo lo computable por uno lo es por el otro. Pero representan dos filosofías opuestas sobre qué es computar.

“This is a paradigm as old as the imperative one. Since the 1930s, beside the Turing Machine, there has existed the λ -calculus, an abstract model for computable functions. Lisp was the first programming language explicitly based on this model.”

— Gabbrielli & Martini, *Programming Languages: Principles and Paradigms*, 2^a ed., cap. 11

1.5.2 1.2 Línea de tiempo de lenguajes funcionales

Año	Lenguaje	Nicho hoy
1958	Lisp	IA simbólica, Emacs Lisp
1973	ML	Compiladores, verificación formal
1975	Scheme	Educación, investigación
1985	Miranda	Origen de Haskell
1986	Erlang	Telecomunicaciones, concurrencia (WhatsApp)
1990	Haskell	Finanzas cuantitativas, criptografía
2003	Scala	Big Data (Spark, Kafka)
2007	Clojure	Concurrencia, blockchain, Datomic
2012	Elixir	Web en tiempo real

¿Por qué los lenguajes imperativos adoptaron el estilo funcional a partir de los 2000s? Cuando los CPUs dejaron de crecer en frecuencia de clock y pasaron

a múltiples núcleos, el estado mutable compartido se volvió el enemigo de la concurrencia. El funcional elimina las condiciones de carrera por diseño. Java 8 (2014), Python, JavaScript ES6 (2015) y TypeScript incorporaron `map`, `filter`, `reduce`, `lambdas/closures` y estructuras inmutables.

“Let us stress, finally, that functional languages had a tremendous impact on the design of programming languages of any paradigm. Many concepts and experimental features in functional programming have later migrated to other paradigms. Among these concepts, type systems, generics, polymorphism, type inference...”

— Gabbrielli & Martini, cap. 11

1.6 Parte II — El Modelo Funcional de Computación

1.6.1 2.1 Dos modelos, dos filosofías

“Conventional languages base their computational model on the transformation of the state. The heart of this model is the concept of modifiable variable.”

— Gabbrielli & Martini, cap. 11

El paradigma **imperativo** computa *modificando el estado*:

estado inicial $\rightarrow$ instrucción1 $\rightarrow$ estado2 $\rightarrow$ instrucción2 $\rightarrow \dots \rightarrow$ resultado

El paradigma **funcional** computa *reescribiendo expresiones*:

expresión $\rightarrow$ β -reducción $\rightarrow$ expresión_reducida $\rightarrow \dots \rightarrow$ forma_normal

“In this chapter, we discuss the functional programming paradigm, where computation proceeds by term rewriting and not through modification of the state. Languages of this paradigm, at least in their ‘pure’ form, do not use the concept of memory.”

— Gabbrielli & Martini, cap. 11

1.6.2 2.2 Ejemplo concreto: mismo problema, dos paradigmas

Problema: Calcular la suma de los cuadrados de los números pares de [1, 2, 3, 4, 5, 6].

Paradigma imperativo:

```
const numeros = [1, 2, 3, 4, 5, 6];
let suma = 0;
for (let i = 0; i < numeros.length; i++) {
  if (numeros[i] % 2 === 0) {
    suma += numeros[i] * numeros[i];
  }
}
console.log(suma); // 56
```

El programa *describió los pasos* para llegar al resultado, modificando suma en cada iteración.

Paradigma funcional:

```
const resultado = [1, 2, 3, 4, 5, 6]
  .filter(n => n % 2 === 0) // [2, 4, 6]
  .map(n => n * n)           // [4, 16, 36]
  .reduce((acc, n) => acc + n, 0); // 56

console.log(resultado); // 56
```

El programa *describió la transformación* que lleva de los datos al resultado. Sin variables mutables, sin estado intermedio nombrado.

Para notar: El mismo resultado, pero el segundo enfoque: - No modifica ninguna variable existente - Cada paso produce una nueva colección (no modifica el array original) - Es independiente del orden de ejecución (paralelizable)

1.6.3 2.3 β -reducción: el mecanismo del λ -cálculo

En Clojure (lenguaje puramente funcional basado en Lisp), el mismo cómputo se escribe:

```
(->> '(1 2 3 4 5 6)
      (filter even?)
      (map #(* % %))
      (reduce + 0))
; => 56
```

El operador `->>` (thread-last) pasa el resultado de cada expresión como último argumento de la siguiente — es la composición de funciones en estilo Lisp.

¿Qué es la β -reducción? Es el paso de evaluación básico del λ -cálculo: dada una aplicación de función $(\lambda x. \text{expresión})$ argumento, se sustituye argumento por x en la expresión. Por ejemplo:

$(\lambda x. x * x) 5 \rightarrow_{\beta} 5 * 5 \rightarrow 25$

Cada aplicación de `.map(n => n * n)` sobre un elemento es, en esencia, una β -reducción.

1.7 Parte III — Los Tres Pilares del Paradigma Funcional

1.7.1 3.1 Pilar 1: Funciones Puras

Definición: Una función es *pura* si su salida depende exclusivamente de sus argumentos y no produce ningún efecto colateral (no modifica variables externas, no hace I/O, no lanza excepciones).

```
// □ IMPURA – lee y modifica estado externo
let contador = 0;
const incrementar = () => {
  contador++; // efecto colateral: modifica variable externa
  return contador; // la salida depende del estado del entorno
};

incrementar(); // → 1
incrementar(); // → 2 (mismo código, resultado distinto)

// □ PURA – sin estado externo
const sumar = (a: number, b: number): number => a + b;

sumar(3, 4); // → 7 (siempre)
sumar(3, 4); // → 7 (siempre)
```

En Clojure:

```
;; □ Pura – mismo resultado siempre
(defn sumar [a b] (+ a b))
(sumar 3 4) ; => 7
```

¿Por qué importa? - **Testeable:** un test de `sumar(3, 4)` que pasa hoy pasará mañana — no hay estado que cambie - **Predecible:** podés razonar sobre la función

sin saber qué hizo el código antes - **Segura en paralelo:** dos hilos que llamen a sumar simultáneamente no interfieren

“One may reason on those components in isolation, with the guarantee that they will always behave in the same manner, since no side-effect is around.”

— Gabbrielli & Martini, cap. 11

1.7.2 3.2 Pilar 2: Inmutabilidad

Definición: Una vez que un valor es asignado, no puede ser modificado. Si necesitas un valor diferente, creás uno nuevo.

```
// □ Estilo imperativo – mutación
let numeros = [1, 2, 3];
numeros.push(4); // modifica numeros en el lugar – mutación oculta
console.log(numeros); // [1, 2, 3, 4] – ¿cuándo cambió?

// □ Estilo funcional – inmutabilidad
const numeros = [1, 2, 3] as const;
const masNumeros = [...numeros, 4]; // nueva colección
console.log(numeros); // [1, 2, 3] – intacto
console.log(masNumeros); // [1, 2, 3, 4]
```

En Clojure toda estructura es inmutable por defecto:

```
(def numeros [1 2 3])           ;; vector inmutable
(def mas-numeros (conj numeros 4)) ;; nueva colección
(println numeros)                ; => [1 2 3] (intacto)
(println mas-numeros)            ; => [1 2 3 4]
```

“In a functional language, a list is an immutable data, thus what, e.g., in Python would be called instead a tuple.”

— Gabbrielli & Martini, cap. 11

¿Por qué const y no let en TypeScript funcional? - let permite reasignar → invita a pensar en mutación → introduce estado - const fuerza a pensar en cada valor como definitivo → obliga al estilo funcional - La restricción no es técnica sino **disciplina de paradigma**

1.7.3 3.3 Pilar 3: Transparencia Referencial

Definición: Una expresión es *referencialmente transparente* si puede ser reemplazada por su valor sin cambiar el comportamiento del programa.

```
// □ Referencialmente transparente
const doble = (n: number): number => n * 2;

// Podemos reemplazar doble(5) por 10 en cualquier contexto:
const resultado1 = doble(5) + doble(5); // → 20
const resultado2 = 10 + 10;              // → 20 (equivalente)

// □ NO referencialmente transparente
let x = 0;
const siguiente = (): number => ++x; // efecto colateral

const r1 = siguiente() + siguiente(); // → 1 + 2 = 3
const r2 = 1 + 1;                    // → 2 ≠ 3 (NO equivalente)
```

“This property, which is immediately falsified when there are side effects, is taken by many authors as the criterion for a pure functional language: a language is purely functional if it satisfies this condition. This is a very important property which makes it possible to reason about a functional program as if it were an algebraic expression.”

— Gabbrielli & Martini, cap. 11

Implication práctica: La transparencia referencial permite al compilador aplicar optimizaciones agresivas (memoización, evaluación lazy, reordenamiento) y permite al programador razonar localmente — cada función es predecible sin contexto del programa completo.

1.8 Parte IV — TypeScript en Modo Funcional

1.8.1 4.1 map — transformar sin mutar

map aplica una función a cada elemento de un array y devuelve un nuevo array. No modifica el original.

Sin map (imperativo):

```
const numeros = [1, 2, 3, 4, 5];
const dobles: number[] = [];
for (let i = 0; i < numeros.length; i++) {
  dobles.push(numeros[i] * 2); // mutación de dobles
}
```

Con map (funcional):

```
const numeros = [1, 2, 3, 4, 5];
const dobles = numeros.map(n => n * 2);
// dobles = [2, 4, 6, 8, 10]
// numeros sigue siendo [1, 2, 3, 4, 5]
```

Tipo de map: `Array<A>.map(f: (a: A) => B): Array<B>` — transforma el tipo de los elementos.

1.8.2 4.2 filter — seleccionar sin mutar

filter devuelve un nuevo array con solo los elementos que satisfacen un predicado.

```
const numeros = [1, 2, 3, 4, 5, 6];
const pares = numeros.filter(n => n % 2 === 0);
// pares = [2, 4, 6]
```

Tipo de filter: `Array<A>.filter(pred: (a: A) => boolean): Array<A>` — no cambia el tipo, solo selecciona.

1.8.3 4.3 reduce — agregar sin mutar

reduce combina todos los elementos de un array en un valor único, aplicando una función acumuladora.

```
const numeros = [1, 2, 3, 4, 5];
const suma = numeros.reduce((acumulador, n) => acumulador + n, 0);
// suma = 15

// Traza de evaluación:
// paso 1: (0, 1) → 1
// paso 2: (1, 2) → 3
```

```
// paso 3: (3, 3) → 6
// paso 4: (6, 4) → 10
// paso 5: (10, 5) → 15
```

reduce es el más general: map y filter pueden implementarse con reduce.

“The function fold(f, init, list_of_data) is a (usually predefined) function that accumulates the elements of list_of_data, returning the accumulated result; init is the initial value used to start the fold. For example, fold(fn x,y => x+y, 0, list_of_int) returns the sum of all the elements of list_of_int.”

— Gabbrielli & Martini, cap. 11

1.8.4 4.4 Composición con pipe

Encadenar map, filter y reduce con métodos de array ya es composición, pero se puede hacer explícita:

```
// pipe: aplica funciones en secuencia (izquierda a derecha)
const pipe = (...fns: Array<(x: any) => any>) =>
  (x: any) => fns.reduce((acc, fn) => fn(acc), x);

const procesarNumeros = pipe(
  (nums: number[]) => nums.filter(n => n % 2 === 0),
  (nums: number[]) => nums.map(n => n * n),
  (nums: number[]) => nums.reduce((acc, n) => acc + n, 0)
);

procesarNumeros([1, 2, 3, 4, 5, 6]); // → 56
```

“Programming in a functional style revolves around immutable data, manipulated by a (large) set of (small) functions. Using higher-order functions one may define general programming schemata as functions, which can then be instantiated to obtain specific program behaviors.”

— Gabbrielli & Martini, cap. 11

1.8.5 4.5 Funciones de Orden Superior

Una función de **orden superior** es aquella que recibe funciones como argumento o devuelve una función como resultado. `map`, `filter` y `reduce` son herramientas de orden superior.

```
// La función hacerDoble es de orden superior: devuelve una función
const multiplicarPor = (factor: number) =>
  (n: number): number => n * factor;

const hacerDoble = multiplicarPor(2);
const hacerTriple = multiplicarPor(3);

hacerDoble(5); // → 10
hacerTriple(5); // → 15

// Uso con map:
[1, 2, 3].map(hacerDoble); // → [2, 4, 6]
[1, 2, 3].map(hacerTriple); // → [3, 6, 9]
```

“The essential point we want to make is that the extensive use of program schemata increases the modularity of code.”

— Gabbrielli & Martini, cap. 11

1.9 Parte V — Clojure: el Funcional Puro

1.9.1 5.1 ¿Qué es Clojure?

Clojure es un dialecto de Lisp que corre en la JVM (Java Virtual Machine). Fue creado por Rich Hickey en 2007 con el objetivo de ser un lenguaje **funcional puro y práctico** para sistemas concurrentes.

Características clave: - Inmutabilidad estructural nativa (no hay forma de mutar un vector o map Clojure “in place”) - Todo es una expresión — no hay sentencias - Sintaxis de listas (prefix notation): (operador argumento1 argumento2) - Interoperabilidad total con Java

¿Por qué lo estudiamos? TypeScript *puede* usarse de forma funcional, pero *no* es un lenguaje funcional puro — nada impide hacer `let` o `push`. Clojure *obliga* el estilo funcional. Ver el mismo problema resuelto en ambos idiomas hace explícitas

las diferencias filosóficas.

1.9.2 5.2 Sintaxis básica de Clojure

```
;; Definir un valor (equivalente de const en TS)
(def nombre "María")

;; Definir una función
(defn saludar [nombre]
  (str "Hola, " nombre "!"))

(saludar "Ana") ; => "Hola, Ana!"

;; Lista literal
'(1 2 3 4 5 6)

;; Vector literal (acceso por índice 0(1))
[1 2 3 4 5 6]

;; Función anónima
(fn [x] (* x x))    ;; forma larga
#(* % %)           ;; forma corta (% = primer argumento)
```

1.9.3 5.3 map, filter, reduce en Clojure

```
;; map
(map #(* % 2) [1 2 3 4 5])
; => (2 4 6 8 10)

;; filter
(filter even? [1 2 3 4 5 6])
; => (2 4 6)

;; reduce
(reduce + 0 [1 2 3 4 5])
```

```
; => 15
```

Thread-last operator (->) — encadena transformaciones de izquierda a derecha:

```
(->> [1 2 3 4 5 6]
  (filter even?)      ; => (2 4 6)
  (map #(* % %))      ; => (4 16 36)
  (reduce + 0))       ; => 56
```

Este código Clojure hace exactamente lo mismo que el `chain .filter().map().reduce()` en TypeScript.

1.9.4 5.4 Comparativa TypeScript vs Clojure

Concepto	TypeScript (multiparadigma)	Clojure (funcional puro)
Valor inmutable	<code>const x = 5</code>	<code>(def x 5)</code>
Función anónima	<code>(n: number) => n * 2</code>	<code>#(* % 2)</code>
Función nombrada	<code>const doble = (n) => n * 2</code>	<code>(defn doble [n] (* 2 n))</code>
map	<code>[1,2,3].map(n => n*2)</code>	<code>(map #(* % 2) '(1 2 3))</code>
filter	<code>arr.filter(n => n > 2)</code>	<code>(filter #(> % 2) coll)</code>
reduce	<code>arr.reduce((a, n) => a + n, 0)</code>	<code>(reduce + 0 coll)</code>
Composición	<code>.filter().map().reduce()</code> o <code>pipe</code>	<code>(->> coll ...)</code>
Lista vacía	<code>[]</code>	<code>'() o []</code>
Mutación	Posible (disciplina del programador)	Imposible (por diseño del lenguaje)

Diferencia filosófica clave: TypeScript dice “*podés ser funcional*”. Clojure dice “*no tenés otra opción*”. La disciplina que en TS es una decisión voluntaria, en Clojure es una garantía del lenguaje.

1.10 Parte VI — Lenguajes Puros vs Multiparadigma

1.10.1 6.1 Lenguajes funcionales puros

“In pure functional languages, there is neither a state nor a modifiable variable. The computation proceeds — at least in principle — by rewriting expressions.”

— Gabbrielli & Martini, cap. 11

Ejemplos de lenguajes funcionales puros: **Haskell**, **Clojure** (para estructuras de datos), **Erlang** (para procesos).

En estos lenguajes: - No existe el concepto de variable mutable - No existe el concepto de instrucción de asignación $x = 5$ (solo ligaduras) - Los efectos de I/O se modelan explícitamente (mónadas en Haskell, refs/atoms en Clojure)

1.10.2 6.2 El funcional en lenguajes multiparadigma

“It is clear that one may use a functional programming style also using programming languages that allow for other programming paradigms. Once a language provides higher-order functions, it becomes easy to write large programs which avoid state-based computations.”

— Gabbrielli & Martini, cap. 11

TypeScript, JavaScript, Python, Kotlin, Scala, Swift y Rust son multiparadigma. En todos se puede escribir código funcional, pero el lenguaje no *obliga* al estilo funcional — es una elección del equipo.

¿Cuándo es preferible el estilo funcional? - Transformaciones de datos (ETL pipelines, validaciones, cálculos) - Lógica de negocio compleja (pocas dependencias externas, fácil de testear) - Componentes concurrentes (sin estado mutable compartido)

¿Cuándo el imperativo es inevitable? - I/O (leer archivos, consultar bases de datos) - Performance crítica con estructuras mutables (videojuegos, simulaciones en tiempo real) - Integración con APIs que devuelven estado mutable

1.11 Puntos Clave del Tema

1. **El paradigma funcional tiene 90 años** — nació del λ -cálculo de Church (1936), antes de que existiera ningún hardware para ejecutarlo.

2. **Dos formas de computar:** imperativo = modificar estado; funcional = reescribir expresiones. Son computacionalmente equivalentes pero filosóficamente opuestos.
 3. **Tres pilares:**
 - **Funciones puras** — mismo input → mismo output, sin efectos colaterales
 - **Inmutabilidad** — los valores no se modifican, se crean nuevos
 - **Transparencia referencial** — una expresión puede reemplazarse por su valor en cualquier contexto
 4. **map, filter, reduce** son los bloques de construcción del estilo funcional en TypeScript. Cada uno devuelve una nueva colección sin modificar la original.
 5. **const sobre let** no es una regla arbitraria — es la expresión de la inmutabilidad en TypeScript. Usar `let` invita al estado mutable.
 6. **Clojure vs TypeScript:** Clojure obliga el funcional por diseño; TypeScript lo habilita pero no lo obliga. Estudiar ambos revela qué es estructura del paradigma y qué es disciplina del programador.
 7. **El funcional entró a los lenguajes imperativos** por necesidad real: concurrencia con múltiples núcleos. El estado mutable compartido genera condiciones de carrera; la inmutabilidad las elimina por diseño.
-

1.12 Autoevaluación

1.12.1 Nivel 1 — Comprensión conceptual

1. ¿Qué significa que la computación en el paradigma funcional “procede por reescritura de expresiones”? Describí con tus palabras cómo se diferencia de la computación imperativa.
2. ¿Qué hace que una función sea “pura”? Identificá cuáles de las siguientes funciones son puras y cuáles son impuras, y justificá:

```
// (a)
const cuadrado = (n: number): number => n * n;

// (b)
let log: string[] = [];
const registrar = (msg: string): void => { log.push(msg); };
```

```
// (c)
const saludar = (nombre: string): string => `Hola, ${nombre}`;

// (d)
const ahora = (): number => Date.now();
```

3. ¿Por qué la transparencia referencial facilita el razonamiento sobre el código? Usá un ejemplo concreto en tu respuesta.

1.12.2 Nivel 2 — Aplicación

4. Dado el array `const productos = [{nombre: "libro", precio: 350}, {nombre: "lapiz", precio: 25}, {nombre: "cuaderno", precio: 180}]`, escribí en TypeScript (usando solo `map`, `filter` y `reduce`): - (a) Un array con solo los productos que cuestan más de \$100 - (b) Un array con los nombres (strings) de todos los productos - (c) La suma total del precio de todos los productos

5. Reescribí el siguiente código imperativo en estilo funcional usando `map`, `filter` y `reduce`:

```
const nums = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3];
let resultado = 0;
for (let i = 0; i < nums.length; i++) {
  if (nums[i] > 4) {
    resultado += nums[i] * nums[i];
  }
}
// resultado = ?
```

6. ¿Qué diferencia hay entre `map` y `reduce` en cuanto al tipo de su resultado?

1.12.3 Nivel 3 — Análisis y argumentación

7. “TypeScript multiparadigma tiene el mejor de los dos mundos — podés ser funcional cuando querés y imperativo cuando necesitás.” ¿Estás de acuerdo? ¿Qué pierde TypeScript respecto a Clojure al ser multiparadigma?

8. ¿Por qué la industria adoptó características funcionales *después* de los 2000s y no antes? ¿Qué cambio tecnológico lo motivó?

9. Explicá con un ejemplo concreto por qué una función pura es más fácil de testear unitariamente que una función con efectos colaterales.

1.12.4 Respuestas de Autoevaluación

□ Ver respuestas (intentá responder antes de mirar)

1. En el paradigma imperativo, el programa modifica el estado de variables paso a paso (`suma = 0; suma += 1; suma += 2...`). En el funcional, el programa *evalúa* una expresión: `f(g(h(datos)))` — cada aplicación de función produce un nuevo valor; no hay estado que se modifique.

2. - (a) cuadrado → **pura** — salida determinada exclusivamente por `n`, sin efectos colaterales - (b) registrar → **impura** — efecto colateral: modifica `log` (array externo) - (c) saludar → **pura** — mismo nombre siempre produce el mismo saludo - (d) ahora → **impura** — el resultado depende del estado externo (el reloj del sistema)

3. Si `doble(3)` siempre devuelve 6, podemos reemplazar mentalmente cualquier `doble(3)` en el código por 6. Esto permite razonar localmente: no hay que rastrear el historial de llamadas o el estado previo del entorno.

4.

```
// (a)
const caros = productos.filter(p => p.precio > 100);
// (b)
const nombres = productos.map(p => p.nombre);
// (c)
const total = productos.reduce((acc, p) => acc + p.precio, 0); // 555
```

5.

```
const resultado = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
  .filter(n => n > 4)           // [5, 9, 6, 5]
  .map(n => n * n)              // [25, 81, 36, 25]
  .reduce((acc, n) => acc + n, 0); // 167
```

6. `map` siempre devuelve un array del mismo largo (transforma cada elemento). `reduce` puede devolver cualquier tipo (número, string, objeto, array de diferente tamaño) — es el operador más general.

7. Argumento a favor: la flexibilidad es real — podés usar `fetch`, `console.log` o `push`

cuando el dominio lo requiere. Argumento en contra: la *garantía* que da Clojure (es imposible mutar) no existe en TS — dos personas en el mismo equipo pueden tener convenciones opuestas. Clojure da certeza; TS da libertad. En sistemas grandes o con muchos colaboradores, la certeza tiene valor.

8. Los CPUs dejaron de crecer en frecuencia (alrededor de 2005) y pasaron a múltiples núcleos. Con estado mutable compartido entre hilos, aparecen condiciones de carrera difíciles de depurar. La inmutabilidad elimina el problema por diseño — dos hilos sobre datos inmutables no pueden interferir. Ese fue el incentivo económico para incorporar el estilo funcional en lenguajes de industria.

9. Una función pura como `sumar(a, b)` se testea con `expect(sumar(3, 4)).toBe(7)` — no hace falta configurar ningún estado previo ni limpiar después. Una función impura como `incrementar()` requiere resetear `contador = 0` antes del test, y el resultado depende de cuántas veces se llamó antes — los tests no son independientes entre sí.

1.13 Glosario

Término	Definición
λ-cálculo	Sistema formal de computación creado por Alonzo Church en 1936. Base teórica del paradigma funcional. Opera mediante abstracción de funciones ($\lambda x. \text{expr}$) y β -reducción.
β-reducción	Paso de evaluación en el λ -cálculo: reemplazar el parámetro x de $(\lambda x. \text{expr})$ arg por arg en expr .
Función pura	Función sin efectos colaterales cuya salida es determinada exclusivamente por sus argumentos.
Efecto colateral	Cualquier modificación del estado de variables externas a la función, operación de I/O, o dependencia del estado del entorno.
Inmutabilidad	Propiedad de un valor que no puede ser modificado después de su creación. En TS se expresa con <code>const</code> y <code>as const</code> .

Término	Definición
Transparencia referencial	Propiedad de una expresión que permite reemplazarla por su valor sin cambiar el comportamiento del programa. Equivalente a decir: sin efectos colaterales.
Función de orden superior	Función que recibe funciones como argumentos o devuelve una función como resultado. Ejemplos: <code>map</code> , <code>filter</code> , <code>reduce</code> .
Paradigma funcional puro	Paradigma de programación donde no existe el concepto de variable mutable. La computación procede por reescritura de expresiones.
Paradigma multiparadigma	Lenguaje que soporta múltiples estilos (imperativo, funcional, orientado a objetos). Ejemplos: TypeScript, Python, Kotlin.
Clojure	Dialecto de Lisp en JVM, funcional puro, con inmutabilidad estructural nativa. Creado por Rich Hickey en 2007.
Lisp	Lenguaje de programación creado por John McCarthy en 1958. Primer lenguaje basado en el λ -cálculo. Padre de la familia de dialectos que incluye Clojure, Scheme y Common Lisp.
map	Función de orden superior que aplica una transformación a cada elemento de un array. Devuelve un nuevo array del mismo tamaño.
filter	Función de orden superior que selecciona elementos de un array según un predicado. Devuelve un nuevo array con $\leq$ elementos.
reduce	Función de orden superior que combina todos los elementos de un array en un único valor usando una función acumuladora.

Término	Definición
pipe	Combinador que encadena funciones de izquierda a derecha: <code>pipe(f, g, h)(x) = h(g(f(x)))</code> .
Thread-last (->>)	Operador de Clojure que encadena expresiones pasando el resultado de cada una como último argumento de la siguiente.
Tesis Church-Turing	Establece que toda función computable por una Máquina de Turing puede ser computada por el λ -cálculo (y viceversa). Los dos modelos son computacionalmente equivalentes.

1.14 Referencias Bibliográficas

1. **Gabbrielli, M. & Martini, S.** (2023). *Programming Languages: Principles and Paradigms* (2ª ed.). Springer. — Capítulo 11: Functional Programming Paradigm. *(Fuente principal de este tema)*
2. **Sebesta, R. W.** (2019). *Concepts of Programming Languages* (12ª ed.). Pearson. — Capítulo 15: Functional Programming Languages.
3. **Apunte de cátedra UNTDF** (2025). *Introducción a la Programación Funcional*. Paradigmas y Lenguajes de Programación — IDEI.
4. **Louden, K. C. & Lambert, K. A.** (2011). *Programming Languages: Principles and Practice* (3ª ed.). Course Technology. — Capítulos sobre paradigma funcional.

Nota sobre las citas: Las citas directas en esta guía están extraídas de Gabbrielli & Martini, cap. 11 (versión en inglés).